

Google's PageRank Algorithm: A Linear Algebra Approach to Web Search

Zonglin Han^{UCD}, Menglin Zou^{UCD}

^{UCD}Department of Mathematics, University of California, Davis

MAT 167: Applied Linear Algebra — Summer Session 2, 2026

🔗 Source Code: https://github.com/zhenmunse/UCD_MAT167

Abstract

We present a self-contained implementation and analysis of Google's PageRank algorithm applied to a 12-node directed web graph and a 15-keyword search corpus. Starting from the raw hyperlink adjacency matrix, we construct a row-stochastic Google matrix, repair dangling nodes, and compute the stationary distribution via both eigendecomposition and the power method. We then introduce the teleportation model with damping factor $\alpha = 0.85$ and compare the resulting rankings. Finally, we build a term-document matrix and demonstrate four Boolean search queries—SINGLE, AND, OR, and BUT NOT—ranked by PageRank importance. All computations are carried out in Python with NumPy; MATLAB-equivalent commentary is provided throughout the code.

Keywords: PageRank, Markov chain, eigenvector, stochastic matrix, search engine, MAT 167

1. Introduction

Before Google, search engines ranked web pages primarily by keyword frequency. A page that mentioned “Nissan” many times would rank higher than one that mentioned it once, regardless of the page's actual authority. In 1998, Larry Page and Sergey Brin introduced PageRank [1], an algorithm that exploits the *link structure* of the web to measure page importance. The core insight is elegantly simple:

A web page is important if other important pages link to it.

This circular definition is resolved by formulating it as an eigenvector problem on a stochastic matrix—a direct application of the linear algebra covered in MAT 167. In this paper we walk through the complete PageRank pipeline on a toy network of 12 web pages (labeled A through L), then combine the ranking with a keyword-based term-document matrix to answer four user-generated search queries.

Section 2 reviews the mathematical foundations. Section 3 details the construction and repair of the Google matrix. Section 4 presents two independent methods for computing the PageRank vector. Section 5 introduces the teleportation model. Section 6 builds the search engine. Section 7 summarizes the findings.

2. Mathematical Foundations

2.1. Markov Chains and Stochastic Matrices

Model a web surfer who, at each step, follows a random outgoing link from the current page. If page i has $L_i > 0$ outgoing links, the probability of transitioning to any linked page j is $G_{ij} = 1/L_i$. Collecting these probabilities into a matrix $G \in \mathbb{R}^{n \times n}$ yields a **row-stochastic matrix**: every entry is non-negative and every row sums to one.

The surfer's long-run behavior is described by a **stationary distribution** $\pi \in \mathbb{R}^n$ satisfying

$$\pi^T = \pi^T G, \quad (1)$$

or equivalently, in the column-vector form used for eigendecomposition,

$$G^T \pi = \pi. \quad (2)$$

Thus π is the eigenvector of G^T corresponding to eigenvalue $\lambda = 1$.

2.2. Dangling Nodes

A **dangling node** is a page with no outgoing links ($L_i = 0$). Its row in G is entirely zero, violating the row-stochastic property. The standard fix is to replace the zero row with a uniform distribution:

$$G_{ij} = \frac{1}{n} \quad \text{for all } j, \quad \text{if } L_i = 0. \quad (3)$$

This models a surfer who, upon reaching a dead end, jumps to any page with equal probability.

2.3. The Teleportation Model

Even after fixing dangling nodes, G may not be **irreducible** (strongly connected) or **aperiodic**, which are required for the Perron–Frobenius theorem to guarantee a unique positive stationary distribution. Google's solution is the **teleportation model**:

$$\tilde{G} = \alpha G + (1 - \alpha)E, \quad E = \frac{1}{n} \mathbf{1}_n \mathbf{1}_n^T, \quad (4)$$

where $\alpha \in (0, 1)$ is the **damping factor** (typically $\alpha = 0.85$). With probability α the surfer follows a link; with probability $1 - \alpha$ the surfer “teleports” to a uniformly random page. The matrix $\tilde{G}$ is strictly positive, hence irreducible and aperiodic, and the Perron–Frobenius theorem guarantees a unique stationary distribution with all positive entries.

2.4. The Power Method

For large networks, direct eigendecomposition is infeasible. The **power method** iteratively converges to the dominant eigenvector:

$$\pi_j^T = \pi_{j-1}^T G, \quad (5)$$

starting from an arbitrary probability vector $\pi_0 = \frac{1}{n} \mathbf{1}_n$. Iteration continues until $\|\pi_j - \pi_{j-1}\|_1 \leq \varepsilon$ for a preset tolerance ε .

2.5. Term–Document Matrix and Search

Given t keywords and n web pages, the **term–document matrix** $T \in \{0, 1\}^{t \times n}$ has $T_{ij} = 1$ if keyword i appears on page j . A search query is encoded as a one-hot vector $\mathbf{q} \in \{0, 1\}^t$. The product $\mathbf{d}^T = \mathbf{q}^T T$ identifies which pages contain the queried keyword. For Boolean queries involving multiple keywords, separate $\mathbf{d}$ vectors are computed and combined with elementwise logic:

- **AND**: both $\mathbf{d}$ vectors are nonzero.
- **OR**: at least one $\mathbf{d}$ vector is nonzero.
- **BUT NOT**: the first is nonzero and the second is zero.

Matching pages are then ranked by their PageRank scores.

3. Constructing the Google Matrix

3.1. The Hyperlink Network

Our network consists of 12 web pages A–L with the following directed edges (from $\rightarrow$ to):

Page	Outgoing links
A	B, D
B	D, E
C	B, A, H
D	E, F
E	D, G, C, H
F	A, D, G, K
G	D, I
H	G, J
I	F, G, K, J
J	I, L
K	I, L
L	(none)

Page L is the only dangling node.

3.2. Raw Google Matrix

Each row of G is constructed by dividing the adjacency row by the number of outgoing links. For example, page A links to B and D, so row A has entries $1/2$ in columns B and D, and zero elsewhere. Page L has no outgoing links, so its row is entirely zero.

The resulting 12×12 matrix G satisfies $\sum_j G_{ij} = 1$ for all $i \neq L$ and $\sum_j G_{L,j} = 0$.

3.3. Repairing the Dangling Node

We replace row L with $\frac{1}{12}$ in every column. After this correction, all row sums equal 1 and G is row-stochastic.

4. Computing PageRank

4.1. Eigendecomposition

We compute the eigenvalues of G^T and identify the eigenvector corresponding to $\lambda = 1$. After ensuring non-negative entries and normalizing so that $\|\pi\|_1 = 1$, we obtain the PageRank vector shown in Table 1.

Page D ranks highest because it receives incoming links from many pages (A, B, E, F, G), several of which are themselves well-connected.

4.2. Power Method Verification

Starting from $\pi_0 = \frac{1}{12}\mathbf{1}_{12}$ and iterating $\pi_j^T = \pi_{j-1}^T G$ until $\|\pi_j - \pi_{j-1}\|_1 \leq 10^{-10}$, convergence is achieved in **35 iterations**. The L_1 difference between the power-method result and the eigendecomposition result is approximately 1.75×10^{-11} , confirming numerical agreement.

Table 1: PageRank vector from the corrected Google matrix G (eigendecomposition). Pages are listed in decreasing order of importance.

Rank	Page	PageRank score
1	D	0.1650
2	I	0.1281
3	F	0.1203
4	G	0.1170
5	E	0.1084
6	L	0.0697
7	K	0.0679
8	J	0.0598
9	A	0.0469
10	H	0.0439
11	B	0.0402
12	C	0.0329

5. Teleportation

We construct $\tilde{G} = 0.85 G + 0.15 E$ where $E = \frac{1}{12} \mathbf{1}_{12} \mathbf{1}_{12}^T$. All entries of $\tilde{G}$ are now strictly positive, and all row sums remain 1.

5.1. PageRank with Teleportation

The new stationary distribution is shown in Table 2.

5.2. Effect of Teleportation on Rankings

Comparing Tables 1 and 2, the top two pages (D and I) and the bottom two (B and C) remain unchanged. The only rank swap is between positions 3 and 4: G and F exchange places. Teleportation redistributes some probability mass from highly-linked pages toward peripheral ones, but the overall structure of the ranking is preserved.

The power method for $\tilde{G}$ converges in **28 iterations** (faster than the 35 for G), with an L_1 difference from the eigendecomposition result of approximately 1.70×10^{-11} .

6. Search Engine

6.1. Term–Document Matrix

We construct a 15×12 binary matrix T with 15 car brands (Alfa Romeo, Audi, Cadillac, Dodge, Hyundai, Jaguar, Lexus, Mazda, Mercedes, Nissan, Porsche, Subaru, Tesla, Toyota, Volkswagen) as rows and pages A–L as columns. The entry $T_{ij} = 1$ if brand i appears on page j .

6.2. Query Results

For each query, we compute the matching pages and rank them using the $\tilde{G}$ PageRank scores. The results are summarized in Table 3.

Query 1: “Nissan” (SINGLE). Five pages contain the keyword Nissan: A, C, F, I, and L. Among these, page I has the highest PageRank (0.1208), followed by F (0.1089).

Table 2: PageRank vector from the teleportation matrix $\tilde{G}$ ($\alpha = 0.85$). Pages are listed in decreasing order of importance.

Rank	Page	PageRank score
1	D	0.1542
2	I	0.1208
3	G	0.1107
4	F	0.1089
5	E	0.1050
6	L	0.0737
7	K	0.0665
8	J	0.0652
9	A	0.0522
10	H	0.0514
11	B	0.0513
12	C	0.0400

Table 3: Search query results ranked by $\tilde{G}$ PageRank.

Query	Matching pages	Ranked order
SINGLE: “Nissan”	A, C, F, I, L	$I > F > L > A > C$
AND: “Mercedes AND Jaguar”	A, B, C, L	$L > A > B > C$
OR: “Toyota OR Porsche”	All 12 pages	$D > I > G > F > E > \dots$
BUT NOT: “Mazda BUT NOT VW”	A, G, J, K, L	$G > L > K > J > A$

Query 2: “Mercedes AND Jaguar” (AND). Four pages contain *both* keywords: A, B, C, and L. Despite being a dangling node in the original network, page L ranks highest (0.0737) because the dangling-node repair and teleportation give it a baseline score, and it receives links from J and K.

Query 3: “Toyota OR Porsche” (OR). Every page in the network contains at least one of these two keywords. The ranking therefore reduces to the global PageRank order: $D > I > G > F > E > L > K > J > A > H > B > C$.

Query 4: “Mazda BUT NOT Volkswagen” (BUT NOT). Six pages contain Mazda (A, C, F, G, J, K, L), but C and F also contain Volkswagen, so they are excluded. The remaining five pages are ranked $G > L > K > J > A$.

7. Summary

We have demonstrated how Google’s PageRank algorithm transforms a directed hyperlink graph into a global importance ranking via the stationary distribution of a Markov chain. The key linear algebra ingredients are:

1. **Stochastic matrix construction:** normalizing the adjacency matrix row-by-row to obtain transition probabilities.

2. **Eigendecomposition:** finding the eigenvector of G^T corresponding to $\lambda = 1$.
3. **Power method:** an iterative alternative that scales to large networks.
4. **Teleportation:** a rank-one perturbation that guarantees convergence and uniqueness via the Perron–Frobenius theorem.

Combined with a term–document matrix, the PageRank vector provides a principled way to rank search results by both keyword relevance and structural importance in the web graph.

Acknowledgments

We thank Professor D. P. Varn for the course materials and project design, and Professor Naoki Saito for the lecture notes on which the SVD and search engine material is based. The project idea is credited to Professor Jesús De Loera, as acknowledged by Professor Varn. The code repository is available at github.com/zhenmunse/UCD_MAT167.

References

- [1] S. Brin and L. Page, “The anatomy of a large-scale hypertextual web search engine,” *Computer Networks and ISDN Systems*, vol. 30, pp. 107–117, 1998.
- [2] C. Moler, *Numerical Computing with MATLAB*, SIAM, 2004. Chapter 2: “Google PageRank.” Available at <https://www.mathworks.com/moler/chapters.html>.
- [3] N. Saito, “Lecture Notes for MAT 167,” University of California, Davis, 2016.
- [4] L. Eldén, *Matrix Methods of Data Mining and Pattern Recognition*, SIAM, 2007.